

2.4 典型实例

(3) 选择第 k 小算法三：分治策略

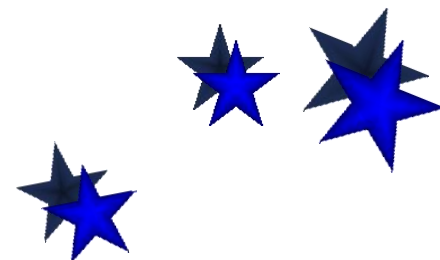
假设元素彼此不等，设计思想：

1. 用某个元素 m^* 作为标准将 S 划分成 S_1 与 S_2 ，其中 S_1 的元素小于 m^* ， S_2 的元素大于等于 m^* 。



2. 如果 $k \leq |S_1|$ ，则在 S_1 中找第 k 小。
如果 $k = |S_1| + 1$ ，则 m^* 是第 k 小。
如果 $k > |S_1| + 1$ ，则在 S_2 中找第 $k - |S_1| - 1$ 小。

算法效率取决于子问题规模



2.4 典型实例

 如何通过 m^* 控制子问题规模? 实例: $n=15, k=6$

(1) 5个数一组，共划分成 $n/5$ 组

8	2	3	5	7	6	11	14	1	9	13	10	4	12	15
---	---	---	---	---	---	----	----	---	---	----	----	---	----	----

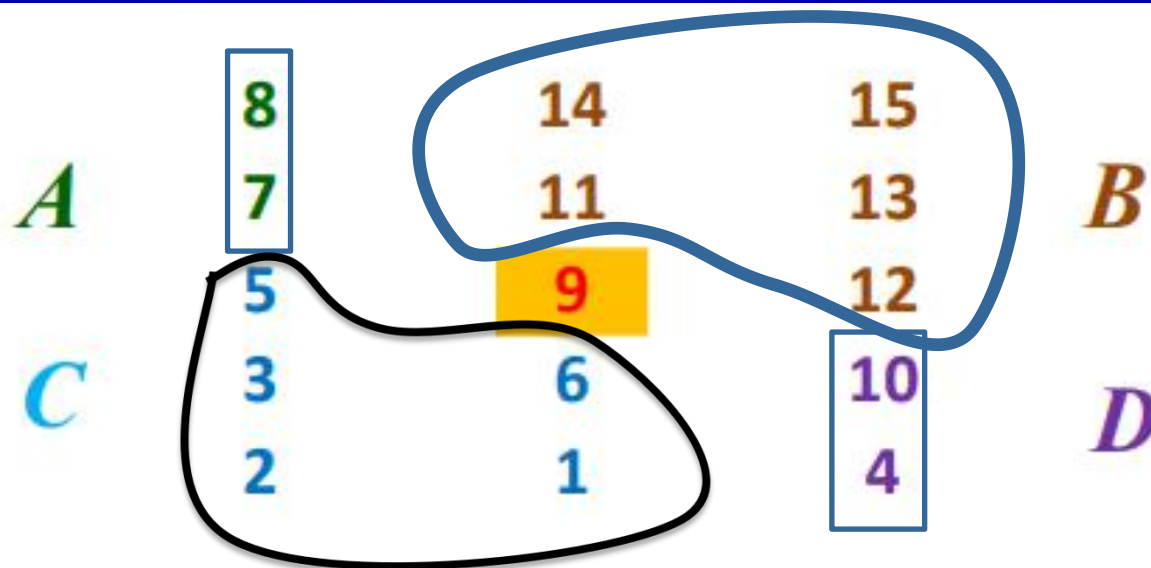
(2) 每组的5个数排序（例如：从大到小），则每组的中位数构成一维数组 M ， M 中共 $n/5$ 个数。

(3) 找到 M 的中位数，该数即为 m^* ，使用 m^* 进行一次划分。

	8	14	15
	7	11	13
M	5	9	12
	3	6	10
	2	1	4

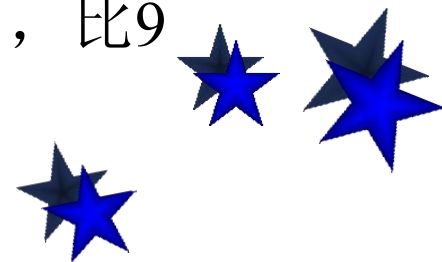
$m^* = 9$ 

2.4 典型实例

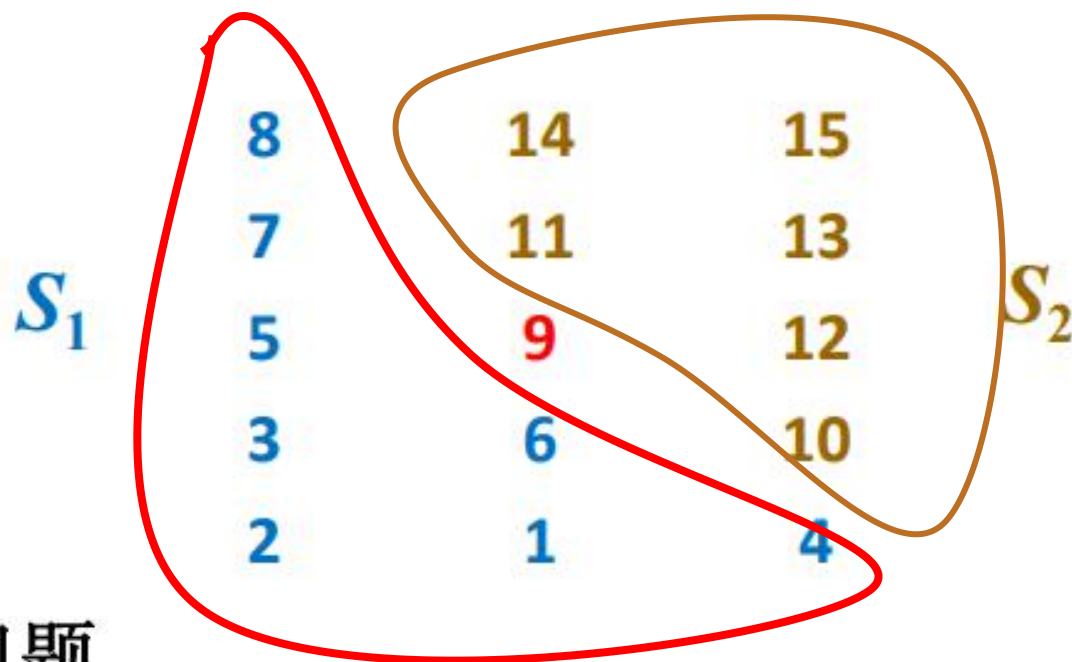


$A(8, 7), D(10, 4)$ 需要与 $m^*(9)$ 比较

归约：将 A 、 D 区域中比 9 小的放入 C (S_1) 中，比 9 大的放入 B (S_2) 中。



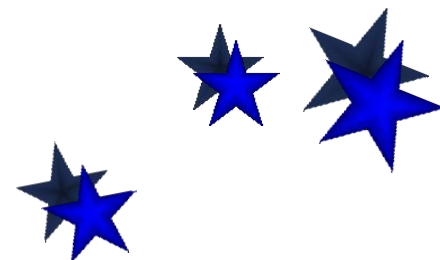
2.4 典型实例



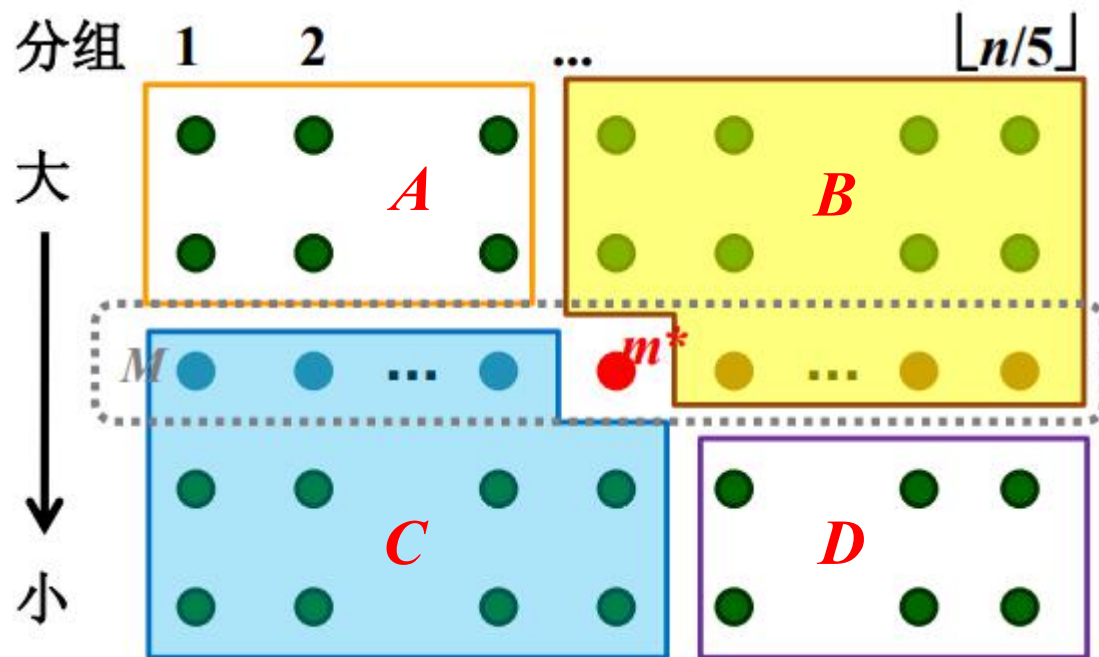
子问题

8 7 5 3 2 6 1 4

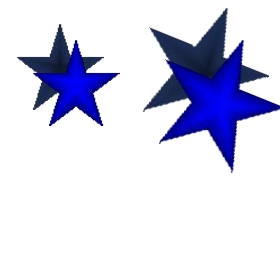
子问题规模 = 8, $k=6$



2.4 典型实例



A : 数需要与 m^* 比大小, B : 数大于 m^*
 C : 数小于 m^* , D : 数需要与 m^* 比大小



2.4 典型实例

选择的分治算法 $\text{Select}(S, k)$

输入：数组 S ，正整数 k

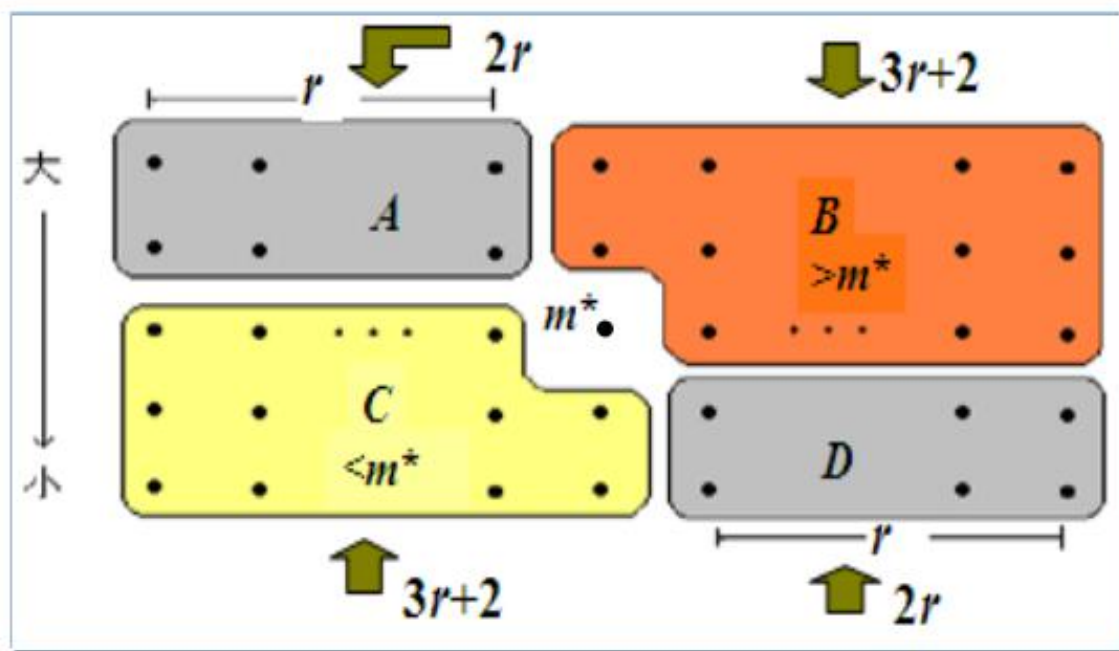
输出： S 中的第 k 小元素

1. 将 S 划分成 5 个一组，共 $\lceil n/5 \rceil$ 个组，组内排序
2. 每组找一个中位数，所有个中位数放到集合 M
3. $m^* \leftarrow \text{Select}(M, \lceil |M|/2 \rceil)$ // 将 S 划分成 A, B, C, D 四个集合
4. 把 A 和 D 的每个元素与 m^* 比较，小的构成 S_1 ，大的构成 S_2
5. $S_1 \leftarrow S_1 \cup C; S_2 \leftarrow S_2 \cup B$
6. if $k = |S_1| + 1$ then 输出 m^*
7. else if $k \leq |S_1|$
8. then $\text{Select}(S_1, k)$
9. else $\text{Select}(S_2, k - |S_1| - 1)$

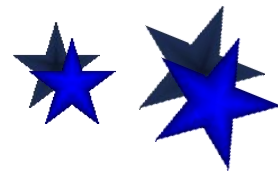


2.4 典型实例

■ 选择问题的算法分析



$n = 5(2r + 1)$, $|A| = |D| = 2r$
子问题规模至多: $2r + 2r + 3r + 2 = 7r + 2$



2.4 典型实例

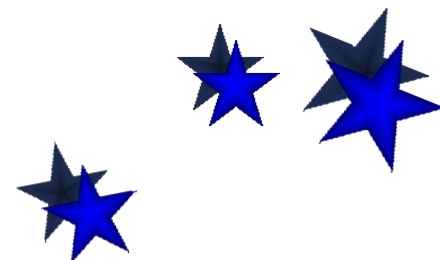
- 子问题规模估计

不妨设 $n = 5(2r + 1)$, $|A| = |D| = 2r$,

$$r = \frac{n/5 - 1}{2} = \frac{n}{10} - \frac{1}{2}$$

划分后子问题规模至多为

$$\begin{aligned} \underline{7r + 2} &= 7\left(\frac{n}{10} - \frac{1}{2}\right) + 2 \\ &= \frac{7n}{10} - \frac{3}{2} < \frac{7n}{10} \end{aligned}$$



2.4 典型实例

- 时间复杂度递推方程

算法工作量 $W(n)$

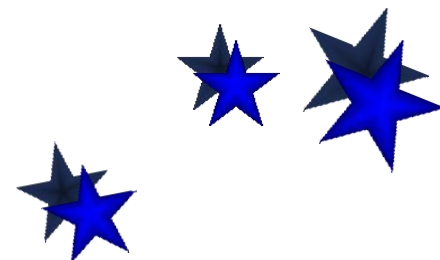
行2: $O(n)$ //每5个数找中位数,构成 M

行3: $W(n/5)$ // M 中找中位数 m^*

行4: $O(n)$ // 用 m^* 划分集合 S

行8-9: $W(7n/10)$ //递归

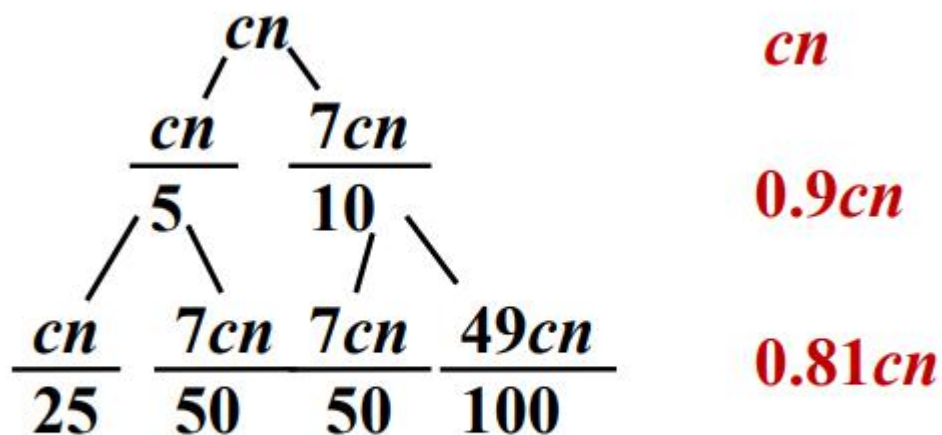
$$W(n) \leq W(n/5) + W(7n/10) + O(n)$$



2.4 典型实例

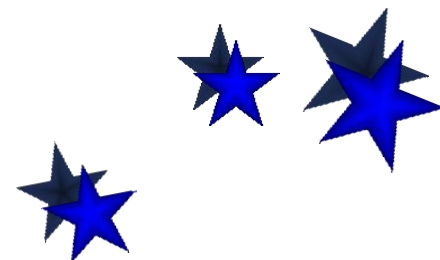
- 时间复杂度递推方程

$$W(n) = W(n/5) + W(7n/10) + cn$$



.....

$$W(n) \leq cn (1 + 0.9 + 0.9^2 + \dots) = O(n)$$

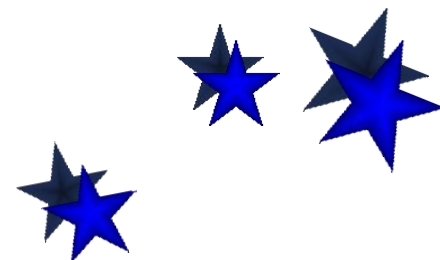


2.4 典型实例

- 讨论：分组时为什么5个元素一组？3个一组或7个一组行不行？

分析：递归调用

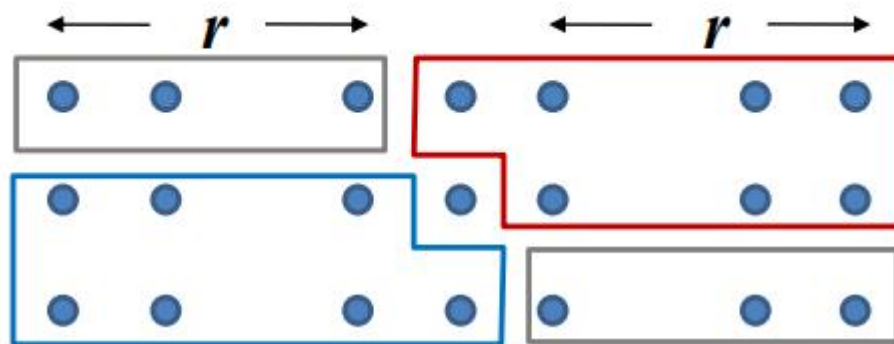
1. 求 m^* 的工作量与 $|M| = n/t$ 相关， t 为每组元素数. t 大, $|M|$ 小
2. 归约后子问题大小与分组元素数 t 有关. t 大, 子问题规模大



2.4 典型实例

- 3分组时的子问题规模

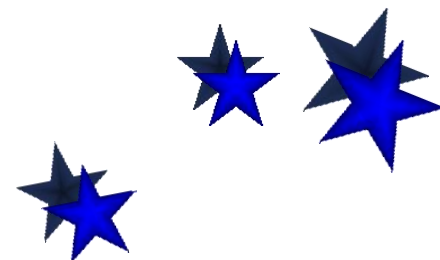
假设 $t=3$, 3个一组:



$$n = 3(2r + 1)$$

$$r = (n/3 - 1)/2 = n/6 - 1/2$$

子问题规模最多为 $4r+1 = 4n/6 - 1$



2.4 典型实例

- 3分组时的时间复杂度

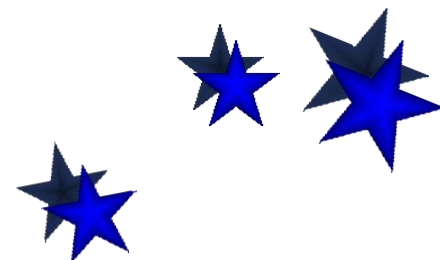
算法的时间复杂度满足方程

$$W(n) = W(n/3) + W(4n/6) + cn$$

由递归树得 $W(n) = \Theta(n \log n)$

关键：

$|M|$ 与归约后子问题规模之和小于 n ，
递归树每行的工作量构成公比小于 1
的等比级数， 算法复杂度才是 $O(n)$.



2.4 典型实例

- 多项式乘法 $C(x) = A(x) B(x)$

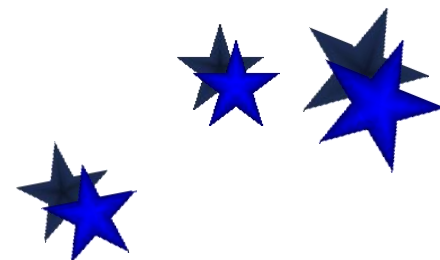
$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{m-1}x^{m-1}$$

$$B(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$$

$$C(x) = \underline{a_0b_0 + (a_0b_1 + a_1b_0)x + \dots} \\ + \underline{a_{m-1}b_{n-1}x^{m+n-2}}$$

其中 x^k 的系数

$$c_k = \sum_{\substack{i+j=k \\ i \in \{0,1,\dots,m-1\} \\ j \in \{0,1,\dots,n-1\}}} a_i b_j, \quad k = 0, 1, \dots, m+n-2$$



2.4 典型实例

- 多项式乘法的应用——向量的卷积计算

(1) 向量的计算

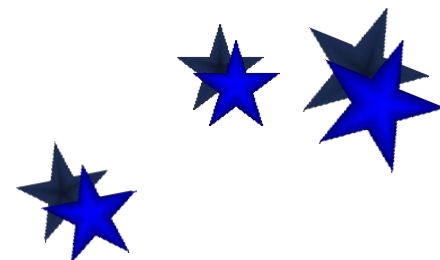
给定向量 $a = (a_0, a_1, \dots, a_{n-1})$
 $b = (b_0, b_1, \dots, b_{n-1})$

向量和 $a+b = (a_0+b_0, a_1+b_1, \dots, a_{n-1}+b_{n-1})$

内积 $a \cdot b = a_0b_0 + a_1b_1 + \dots + a_{n-1}b_{n-1}$

卷积 $a*b = (c_0, c_1, \dots, c_{2n-2})$, 其中

$$c_k = \sum_{\substack{i+j=k \\ i,j < n}} a_i b_j, \quad k = 0, 1, \dots, 2n-2$$

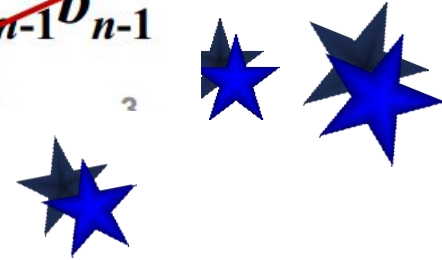


2.4 典型实例

(2) 卷积的含义

- 矩阵中，每个斜线的项之和恰好是卷积中的各个分量

$$\begin{array}{ccccccc}
 & & ab_0 & ab_1 & \cdots & ab_{n-2} & ab_{n-1} \\
 a_0b & \cancel{a_0b_0} & a_0b_1 & \cdots & a_0b_{n-2} & \cancel{a_0b_{n-1}} & \\
 a_1b & \cancel{a_1b_0} & a_1b_1 & \cdots & \cancel{a_1b_{n-2}} & a_1b_{n-1} & \\
 \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \\
 a_{n-2}b & a_{n-2}b_0 & \cancel{a_{n-2}b_1} & \cdots & a_{n-2}b_{n-2} & \cancel{a_{n-2}b_{n-1}} & \\
 a_{n-1}b & \cancel{a_{n-1}b_0} & a_{n-1}b_1 & \cdots & \cancel{a_{n-1}b_{n-2}} & \cancel{a_{n-1}b_{n-1}} & \\
 & & & & c_{2n-3} & c_{2n-2} &
 \end{array}$$





2.4 典型实例

- 或滑动窗口内对应数据项的乘积之和

							a_0	a_1	a_2	a_3	$\dots$	a_{n-3}	a_{n-2}	a_{n-1}
b_{n-1}	b_{n-2}	b_{n-3}	$\dots$	b_3	b_2	b_1	b_0	$c_0 = a_0 b_0$						

						a_0	a_1	a_2	a_3	$\dots$	a_{n-3}	a_{n-2}	a_{n-1}
b_{n-1}	b_{n-2}	b_{n-3}	$\dots$	b_3	b_2	b_1	b_0	$c_I = a_0 b_1 + a_1 b_0$					

$$\mathbf{c}_{n-1} = a_0 \mathbf{b}_{n-1} + a_1 \mathbf{b}_{n-2} + \dots + a_{n-1} \mathbf{b}_0$$

$\mathbf{a}_0$	$\mathbf{a}_1$	$\mathbf{a}_2$	$\mathbf{a}_3$	$\dots$	$\mathbf{a}_{n-3}$	$\mathbf{a}_{n-2}$	$\mathbf{a}_{n-1}$
$\mathbf{b}_{n-1}$	$\mathbf{b}_{n-2}$	$\mathbf{b}_{n-3}$	$\dots$	$\mathbf{b}_3$	$\mathbf{b}_2$	$\mathbf{b}_1$	$\mathbf{b}_0$

$$c_n = a_1 b_{n-1} + a_2 b_{n-2} + \dots + a_{n-1} b_1$$

Diagram illustrating a 1D chain structure with two rows of sites. The top row consists of orange sites labeled $a_0, a_1, a_2, a_3, \dots, a_{n-3}, a_{n-2}, a_{n-1}$. The bottom row consists of green sites labeled $b_{n-1}, b_{n-2}, b_{n-3}, \dots, b_3, b_2, b_1, b_0$. A blue star is positioned above the site a_{n-1} , and a green star is positioned above the site b_0 . A blue arrow points from a_{n-1} to b_{n-1} , and a green arrow points from b_{n-1} to a_{n-2} . The equation $c_{2n-1} = a_{n-1} b_{n-1}$ is written between the two rows.



2.4 典型实例

(3) 向量计算实例

向量 $a = (1, 2, 4, 3)$, $b = (4, 2, 8, 0)$

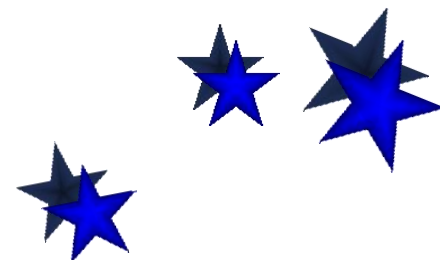
$$a+b = (5, 4, 12, 3)$$

$$a \cdot b = 1 \times 4 + 2 \times 2 + 4 \times 8 + 3 \times 0 = 40$$

$$a * b = (4, 10, 28, 36, 38, 24, 0)$$

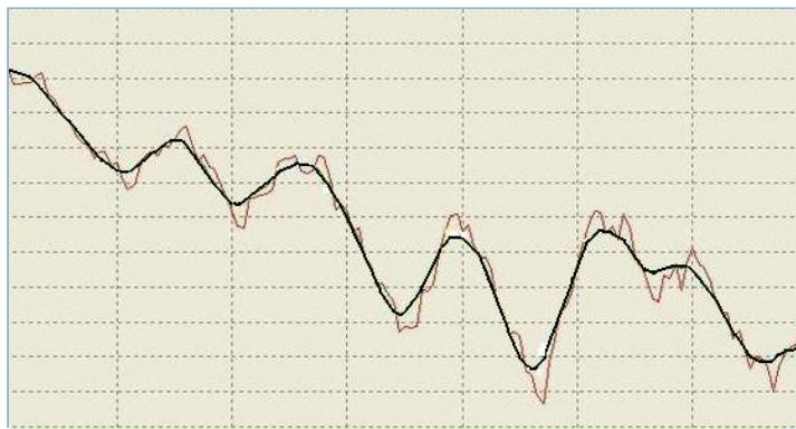
	ab_0	ab_1	ab_2	ab_3
a_0b	1×4	1×2	1×8	1×0
a_1b	2×4	2×2	2×8	2×0
a_2b	4×4	4×2	4×8	4×0
a_3b	3×4	3×2	3×8	3×0

$$c_2 = 4 \times 4 + 2 \times 2 + 1 \times 8 = 28$$



2.4 典型实例

(4) 卷积的应用——信号平滑处理（去除噪音）



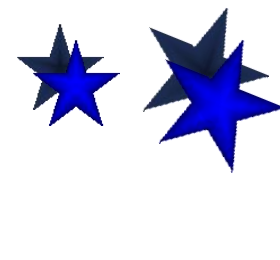
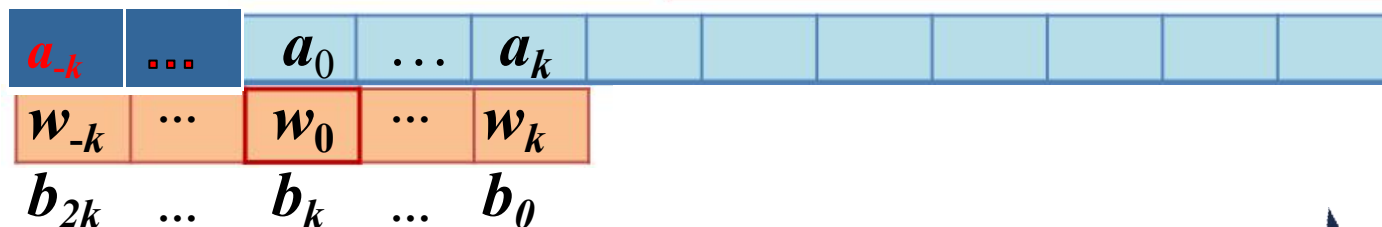
原始信号: $a=(a_0, a_1, \dots, a_{m-1})$

权值向量（步长为 k ，长度 $2k+1$ ）

$$b=(b_{2k}, b_{2k-1}, \dots, b_0) \\ = (w_{-k}, \dots, w_k)$$

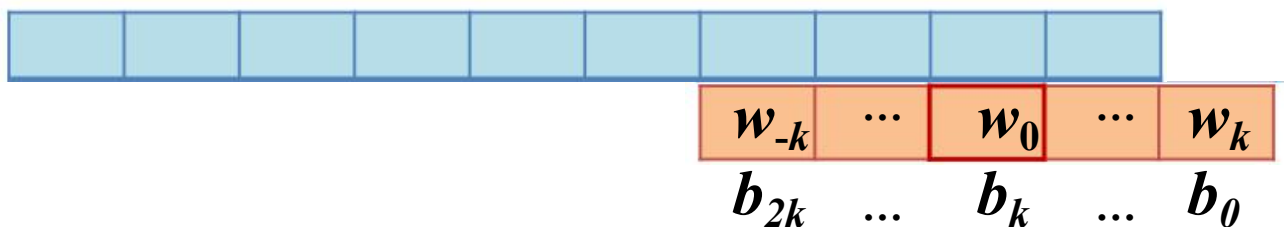
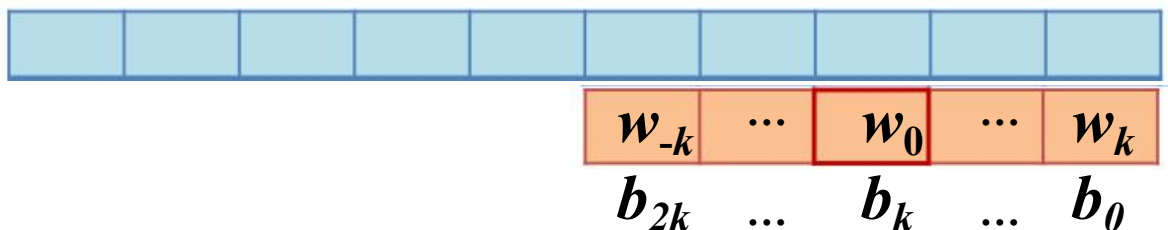
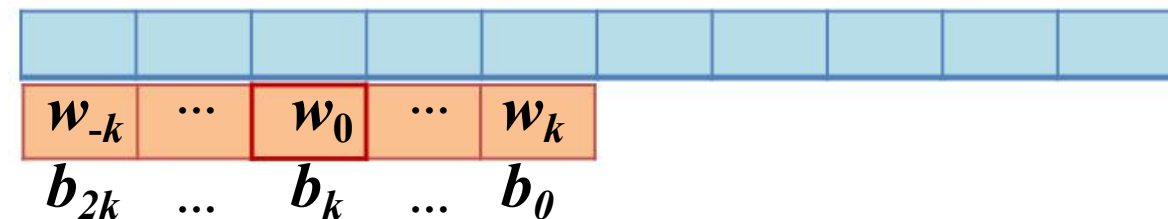
– 将 w_0 与 a_i 对齐计算 a_i'

$$a_i' = \sum_{s=-k}^k a_{i+s} b_{k-s} = \sum_{s=-k}^k a_{i+s} w_s$$

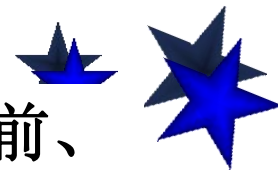


2.4 典型实例

- 当 w_0 与 a_k 对齐时, 才能计算出准确的 a_i'



把向量 b 在 a 上移动计算 $a*b$, 得到 $(a_0', a_1', \dots, a_{m-1}')$ 。前、后 k 个项会有误差。



2.4 典型实例

(4) 卷积计算实例

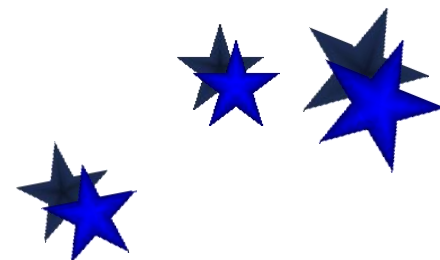
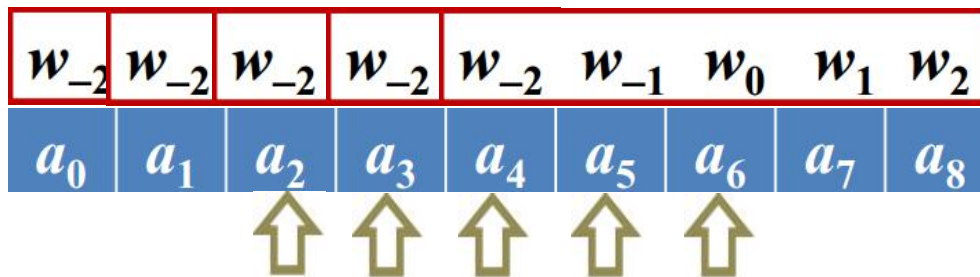
信号向量: $a = (a_0, a_1, \dots, a_8)$

步长: $k = 2$

权值: $w = (w_{-2}, w_{-1}, w_0, w_1, w_2)$
 $= (b_4, b_3, b_2, b_1, b_0)$

$$a_i' = a_{i-2}b_4 + a_{i-1}b_3 + a_ib_2 + a_{i+1}b_1 + a_{i+2}b_0$$

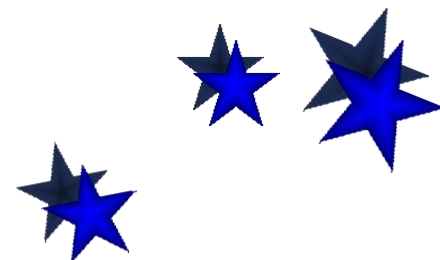
下标之和为 $i + k$



2.4 典型实例

$$a_i' = a_{i-2}b_4 + a_{i-1}b_3 + a_ib_2 + a_{i+1}b_1 + a_{i+2}b_0$$

a_0b_0	a_0b_1	a_0b_2	a_0b_3	a_0b_4	a_2'
a_1b_0	a_1b_1	a_1b_2	a_1b_3	a_1b_4	a_3'
a_2b_0	a_2b_1	a_2b_2	a_2b_3	a_2b_4	a_4'
a_3b_0	a_3b_1	a_3b_2	a_3b_3	a_3b_4	a_5'
a_4b_0	a_4b_1	a_4b_2	a_4b_3	a_4b_4	a_6'
a_5b_0	a_5b_1	a_5b_2	a_5b_3	a_5b_4	
a_6b_0	a_6b_1	a_6b_2	a_6b_3	a_6b_4	
a_7b_0	a_7b_1	a_7b_2	a_7b_3	a_7b_4	
a_8b_0	a_8b_1	a_8b_2	a_8b_3	a_8b_4	



2.4 典型实例

- 求解多项式乘积的算法

(1) 算法1：蛮力算法

$$A(x)=a_0+a_1x+a_2x^2+\dots+a_{n-1}x^{n-1}$$

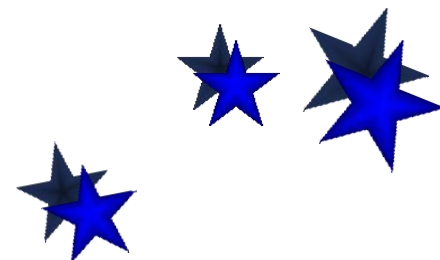
$$B(x)=b_0+b_1x+b_2x^2+\dots+b_{n-1}x^{n-1}$$

$$C(x)=A(x)B(x)$$

$$=a_0b_0 + (a_0b_1+a_1b_0)x + \dots + a_{n-1}b_{n-1}x^{2n-2}$$

$C(x)$ 的系数向量就是 $a*b$.

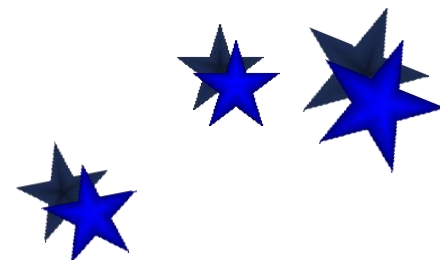
蛮力算法的时间： $O(n^2)$



2.4 典型实例

(2) 算法2：快速傅立叶变换FFT

1. 选择值 $x_0, x_1, \dots, x_{2n-1}$,
2. for $j \leftarrow 0$ to $2n-1$ do
3. $A_j = A(x_j)$
4. $B_j = B(x_j)$
5. $d_j = A_j B_j$
6. 构造多项式 $D(x) = d_0 + d_1 x^1 + \dots + d_{2n-1} x^{2n-1}$



2.4 典型实例

7. for $j \leftarrow 0$ to $2n-1$ do

8. $D_j = D(x_j)$

9. $c_0 = D_1 / 2n$

10. for $j \leftarrow 1$ to $2n-1$ do

11. $c_j = D_{2n-j} / 2n$

多项式插值

➤ 重点研究问题

- (1) 如何选取 $2n$ 个数值($x_0, x_1, x_2, x_3, \dots, x_{2n-1}$)
- (2) 算法中求 $2n$ 个 $A(x_j)$ 、 $B(x_j)$ 、 $D(x_j)$ ，如何高效求得多项式在某个 x 上的值？



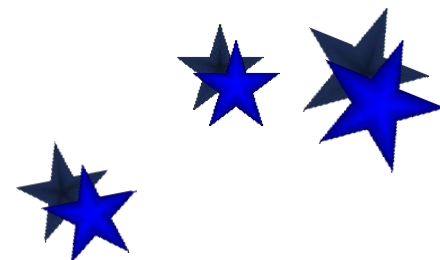
2.4 典型实例

- 重点问题1：如何选取 $2n$ 个数值($x_0, x_1, x_2, x_3, \dots, x_{2n-1}$)
(1) 选取方法：1的 $2n$ 次根

$$x_j = e^{\frac{2\pi j}{2n}i} = e^{\frac{\pi j}{n}i} = \cos \frac{\pi j}{n} + i \sin \frac{\pi j}{n}$$

$$j=0,1,\dots,2n-1, \quad i=\sqrt{-1}$$

x_j 为单位圆上均匀分布的 $2n$ 个点



2.4 典型实例

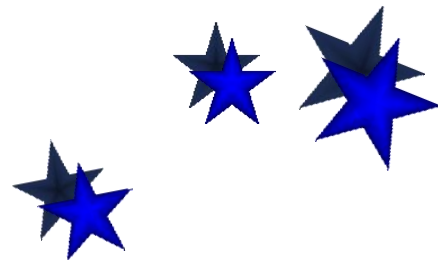
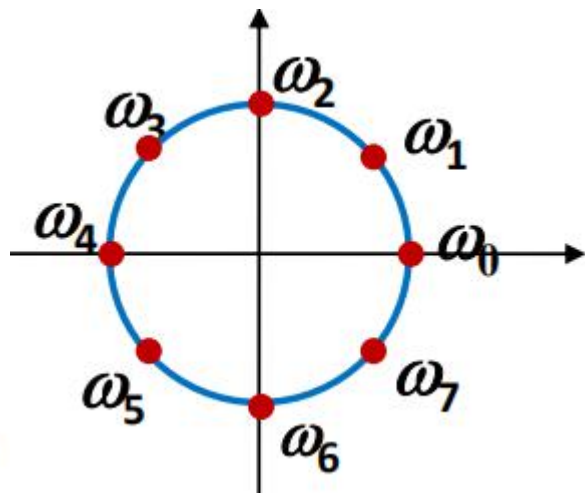
$n=4$ 的实例

$$\omega_0 = 1, \quad \omega_1 = e^{\frac{\pi}{4}i} = \sqrt{2}/2 + \sqrt{2}/2 \cdot i,$$

$$\omega_2 = e^{\frac{\pi}{2}i} = i, \quad \omega_3 = e^{\frac{3\pi}{4}i} = -\sqrt{2}/2 + \sqrt{2}/2 \cdot i,$$

$$\omega_4 = e^{\pi i} = -1, \quad \omega_5 = e^{\frac{5\pi}{4}i} = -\sqrt{2}/2 - \sqrt{2}/2 \cdot i,$$

$$\omega_6 = e^{\frac{3\pi}{2}i} = -i, \quad \omega_7 = e^{\frac{7\pi}{4}i} = \sqrt{2}/2 - \sqrt{2}/2 \cdot i$$



2.4 典型实例

- 重点问题2：算法中求 $2n$ 个 $A(x_j)$ 、 $B(x_j)$ 、 $D(x_j)$ ，如何**高效**求得多项式在某个 x 上的值？

(1) 方法1：蛮力算法

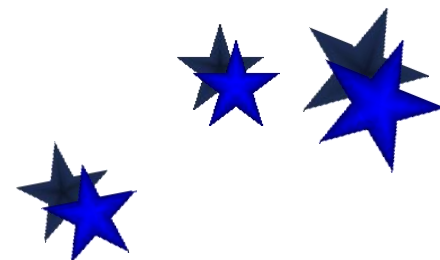
算法1：对每个 x 做下述运算：

依次计算每个项 $a_i x^i$, $i=1, \dots, n-1$

对 $a_i x^i$ ($i=0, 1, \dots, n-1$) 求和.

蛮力算法的时间复杂度

$$T_1(n) = O(n^3)$$



2.4 典型实例

(2) 方法2: 霍纳(Horner)法则

算法2: 依次对每个 x 做下述计算

$$\underline{A_1(x)} = a_{n-1}$$

$$A_2(x) = a_{n-2} + x \underline{A_1(x)} = a_{n-2} + a_{n-1}x$$

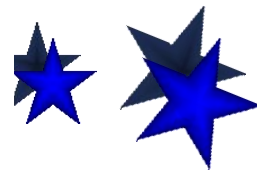
$$A_3(x) = a_{n-3} + x A_2(x) = a_{n-3} + a_{n-2}x + a_{n-1}x^2$$

...

$$A_n(x) = a_0 + x A_{n-1}(x) = A(x)$$

时间复杂度

$$T_2(n) = O(n^2)$$



2.4 典型实例

(3) 方法3：分治法——快速傅里叶变换算法中使用

一般公式 (n 为偶数)

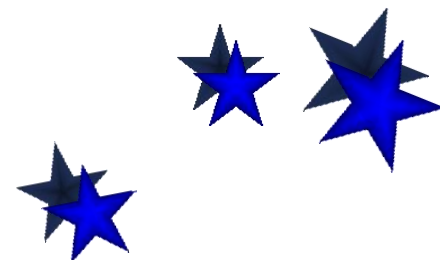
$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$$

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{n-2}x^{(n-2)/2}$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{n-1}x^{(n-2)/2}$$

$$A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$$

偶次数与奇次数多项式计算作为 $n/2$ 规模的子问题，然后利用子问题的解 $A_{\text{even}}(x^2)$ 与 $A_{\text{odd}}(x^2)$ 得到 $A(x)$



2.4 典型实例

分治策略算法描述

1. 计算 1 的所有的 $2n$ 次根

$$1, \omega_1, \omega_2, \dots, \omega_{2n-1}$$

2. for $j \leftarrow 0$ to $2n-1$ do

3. $A_{\text{even}}(w_j^2)$

4. $A_{\text{odd}}(w_j^2)$

5. $A(x) = A_{\text{even}}(w_j^2) + w_j A_{\text{odd}}(w_j^2)$

w_j^2 不需要重新计算，在单位圆上隔一个取一个即可，即取 n 个点，算2圈



2.4 典型实例

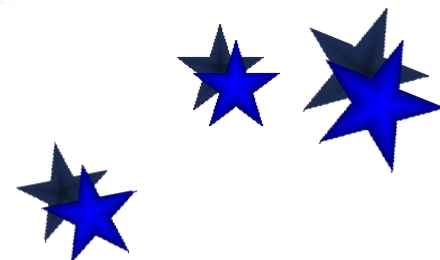
- 分治法多项式求值的时间复杂度分析

$T(n) = T_1(n) + f(n)$ $f(n) = O(n)$ 是步 1 计算 $2n$ 次根的时间

递归过程 $T_1(n) = 2T_1(n/2) + g(n)$ $g(n)$ 是将奇数部和偶数部组合成解的时间
 $T_1(1) = O(1),$

$$T_1(n) = O(n \log n)$$

$$T(n) = O(n \log n) + O(n) = O(n \log n)$$



2.4 典型实例

- FFT算法的时间复杂度分析

步1: 求多项式值 $a_j=A(x_j)$ 和 $b_j=B(x_j)$ $O(n\log n)$

步2: 计算所有的 $d_j=a_j*b_j$ $O(n)$

步3: 构造多项式 $D(x)$

步4: 计算多项式值 $D_j=D(x_j)$ $O(n\log n)$

步5: 利用 D_j 计算所有的 c_j $O(n)$

算法时间为 $O(n\log n)$

